

muduo 网络库

peanutixx

2026 年 4 月 24 日

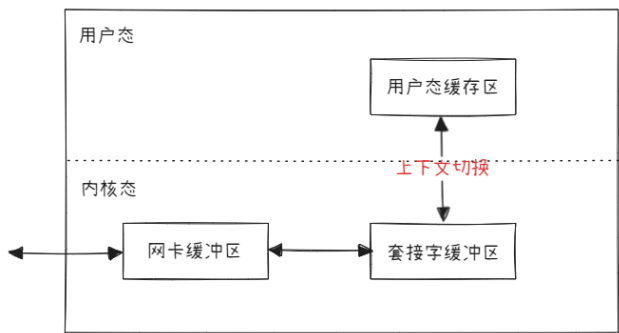
Contents

1 回顾 TCP 编程	1
1.1 网络数据传输过程	1
1.2 五种网络 I/O 模型	2
1.3 I/O 多路复用	5
1.4 TCP 编程的三个半事件	8
2 Reactor 模式	9
2.1 简介	9
2.2 分类	9
2.3 单 Reactor 单线程	9
2.4 单 Reactor 多线程	10
2.5 主从 Reaactor 多线程 (Multi-Reactor)	11
3 muduo 网络库	12
3.1 简介	12
3.2 安装	12
3.3 daytime 服务器	13
3.4 echo 服务器	16
3.5 chat 服务器	19

1 回顾 TCP 编程

1.1 网络数据传输过程

操作系统为了系统的稳定与安全，将 CPU 运行状态划分为用户态和内核态。应用程序通常运行在用户态，当需要执行 I/O 等底层操作时，必须通过系统调用切换到内核态，由内核完成相应操作。



服务器从网络接收数据的流程如下（发送数据的流程刚好相反）：

- 数据经由网络抵达网卡缓冲区。
- 操作系统将网卡缓冲区的数据拷贝到对应的套接字缓冲区（TCP 协议层依据端口做 I/O 解复用）。
- 应用程序将套接字缓冲区的数据拷贝到自己的应用态缓冲区，之后便可使用这些数据。

整个过程的核心就是**两次拷贝**操作，而下面即将介绍的五种网络 I/O 模型，其核心差异恰恰体现在这两个拷贝阶段与应用程序的交互方式上。

1.2 五种网络 I/O 模型

两组重要的概念

1. 阻塞/非阻塞

阻塞/非阻塞的关注点是：用户态进程/线程，在发起一个可能无法立即完成的操作（如读取数据）时，当前进程/线程是否会被挂起。

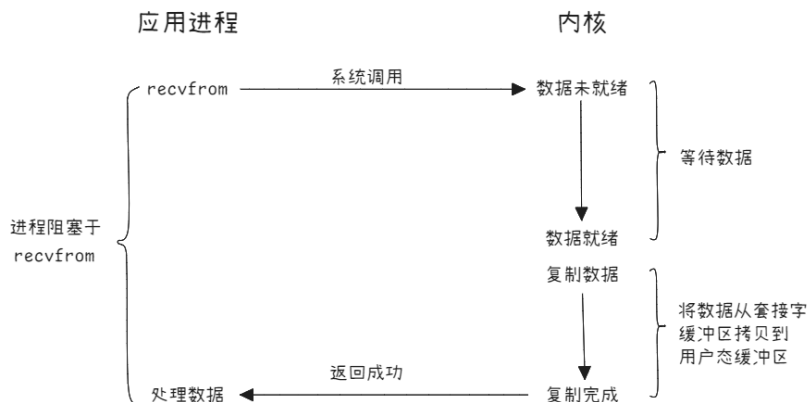
- 阻塞：进程/线程会被挂起（进入阻塞状态），等待事件就绪。
- 非阻塞：进程/线程不会被挂起，该操作会立即返回一个“资源暂不可用”的状态码（EWOULDBLOCK 或 EAGAIN）。线程可以继续执行其他任务，或轮询检查“资源是否可用”。

2. 同步 I/O 和异步 I/O

同步 I/O 和异步 I/O 的关注点是：在拷贝数据时（套接字缓冲区 → 用户态缓冲区），应用程序是否会阻塞。

- 同步 I/O：在拷贝数据时（套接字缓冲区 → 用户态缓冲区），应用程序会阻塞。
- 异步 I/O：在拷贝数据时（套接字缓冲区 → 用户态缓冲区），应用程序不会阻塞。

阻塞 I/O

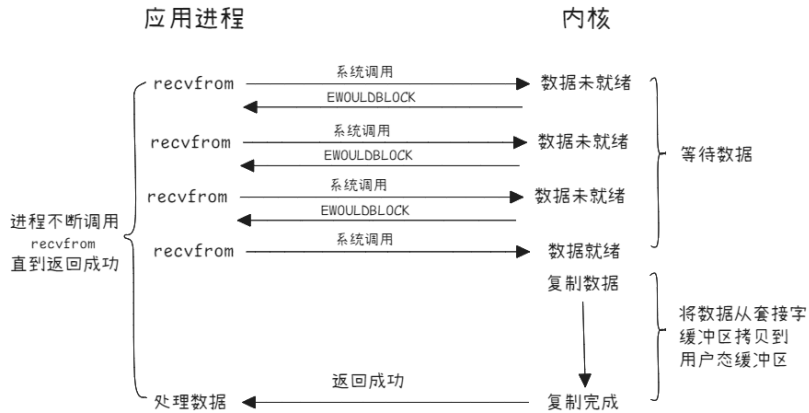


流程：

1. 应用进程调用 `recvfrom`，读取数据。
2. 等待数据就绪（应用进程阻塞）。
3. 将数据从套接字缓冲区拷贝到用户态缓冲区（应用进程阻塞）。
4. 拷贝完成，返回成功（应用进程继续执行）。

在等待数据就绪以及拷贝数据的过程中，应用进程都处于阻塞状态。这种 I/O 方式，我们称之为阻塞 I/O。默认情况下所有的套接字都是阻塞 I/O。

非阻塞 I/O

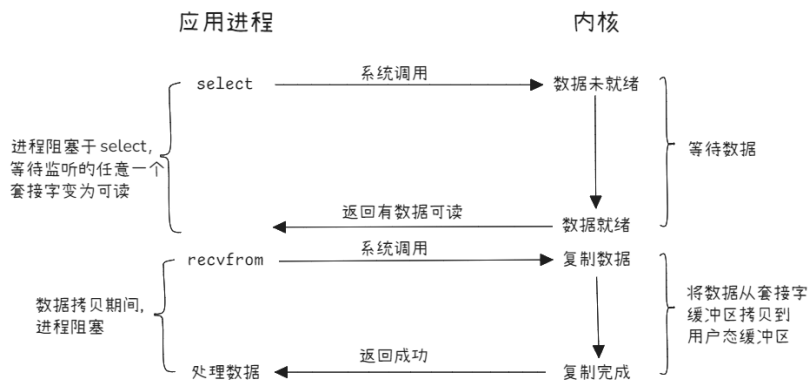


流程：

1. 应用进程调用 `recvfrom`，读取数据。
2. 没有数据就绪，立刻返回 `EWOULDBLOCK` 或 `EAGAIN` 错误码（应用进程不阻塞）。
3. 轮询：应用进程不断地调用 `recvfrom`，询问数据是否就绪（应用进程不阻塞）。
4. 当数据就绪时，将数据从套接字缓冲区拷贝到用户态缓冲区（应用进程阻塞）。
5. 拷贝完成，返回成功。（应用进程继续执行）

非阻塞 I/O 在读取数据报时，若数据没有就绪，会立即返回 `EWOULDBLOCK` 或 `EAGAIN` 错误码，不会让应用进程阻塞等待。应用进程需不断询问内核数据是否就绪，这种方式我们称之为轮询（polling）。轮询非常消耗 CPU 资源。

I/O 多路复用



1. 应用进程调用 `select`，监听多个文件描述符。
2. 等待任意一个文件描述符数据就绪（应用进程阻塞）。
3. 将数据从套接字缓冲区拷贝到用户态缓冲区（应用进程阻塞）。
4. 拷贝完成，返回成功（应用进程继续执行）。

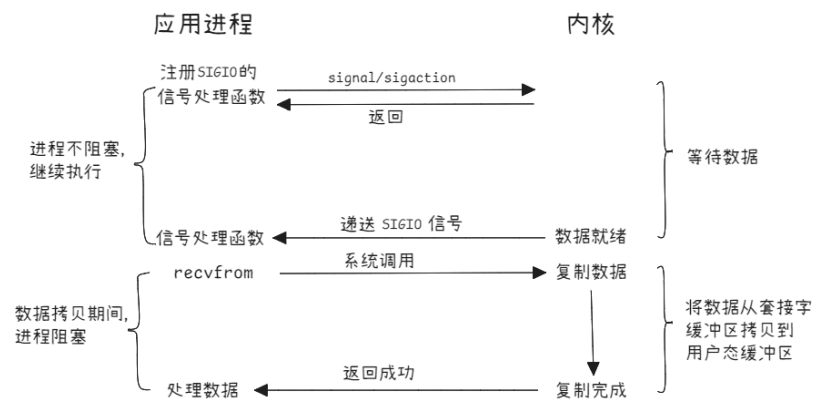
使用 I/O 多路复用时，我们可能会阻塞在 `select`，`poll` 和 `epoll` 等系统调用，而不会阻塞在 I/O 读写操作上。**I/O 多路复用强大的地方在于，它可以用单个线程监听成千上万个文件描述符。只要其中有一个文件描述符就绪，`select` 等函数就会返回。接下来的 I/O 读写操作就不会阻塞了。**

阻塞 I/O + 多线程

阻塞 I/O + 多线程的方式，非常类似 I/O 多路复用。一个线程负责一个文件描述符的读写操作，就算这个线程阻塞了，其它线程依然可以正常工作。不过这种方式会消耗很多系统资源。

信号驱动 I/O (了解)

我们也可以使用 SIGIO 信号，告诉内核：当 I/O 事件就绪时就通知应用程序，然后由应用程序去处理 I/O 事件。这种模型我们称之为信号驱动 I/O 模型。

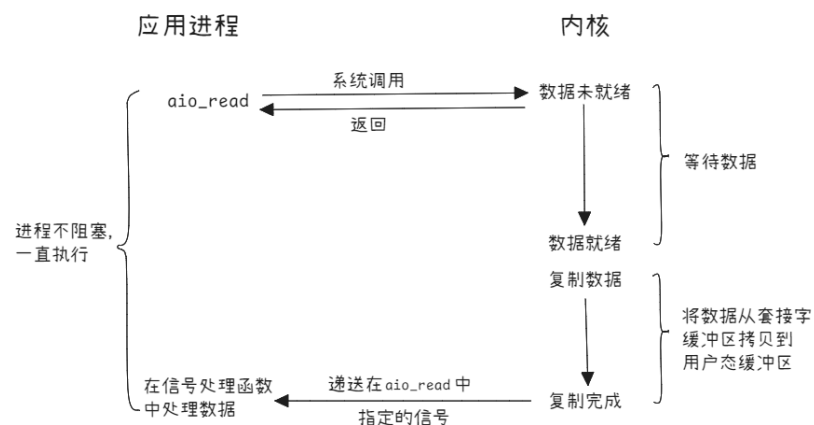


Note

信号驱动 I/O 一般用得很少，因为它引入了复杂的信号机制，增加了程序的不确定性。

异步 I/O (了解)

异步 I/O 和信号驱动 I/O 不同。异步 I/O 会让内核拷贝数据，当数据拷贝完成后再通知应用程序；而信号驱动 I/O，仅仅是让内核通知数据就绪，然后由应用程序自己拷贝数据。

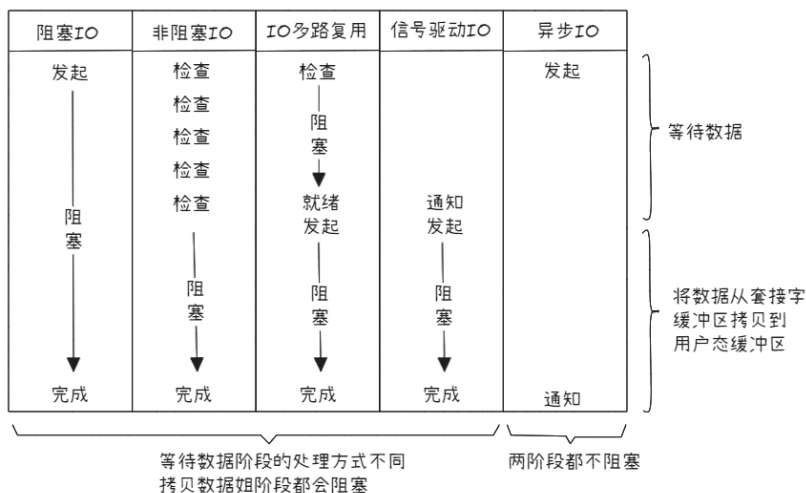


Note

异步 I/O 是堪称的“大杀器”。但遗憾的是，直到 2019 年，Jens Axboe 创造了 `io_uring`^a，Linux 才支持套接字的异步 I/O。这也是为什么在 Linux 平台上，我们一般都是使用 `epoll` + 非阻塞 I/O 来构建高性能高并发网络程序的根本原因。

五种 IO 模型对比

如下图所示，前四种 I/O 模型都需要应用程序自己拷贝数据（同步 I/O），它们的不同之处仅在于等待数据阶段。而异步 I/O 和前四种都不同，等待数据和拷贝数据都由内核完成，应用程序在这两个阶段都不会阻塞。



1.3 I/O 多路复用

我们先回顾一下 Linux 阶段学过的 TCP 编程。下面的代码实现了一个 Echo 服务器，它会把客户端发过来的字母转换成大写，然后再返回给客户端。

EchoServer.cc

```
1 #include "common.h"
2 #include <iostream>
3 #include <netinet/in.h>
4 #include <sys/epoll.h>
5 #include <unistd.h>
6
7 using namespace std;
8
9 static const int BACKLOG = 128;
10 static const int MAXEVENTS = 1024;
11
12 int tcp_listen(uint16_t port)
13 {
14     // 1. 创建主动套接字
15     int sockfd = socket(AF_INET, SOCK_STREAM, 0);
16     if (sockfd == -1) {
17         perror("socket()");
18         exit(-1);
19     }
20     // 2. 设置地址复用
21     int opt = 1;
```

```

22     setsockopt(sockfd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));
23     // 3. 绑定套接字地址
24     struct sockaddr_in addr;
25     bzero(&addr, sizeof(addr));
26     addr.sin_family = AF_INET;
27     addr.sin_port = htons(port);
28     addr.sin_addr.s_addr = htonl(INADDR_ANY); // 通配符地址
29     int err = bind(sockfd, (struct sockaddr*)&addr, sizeof(addr));
30     if (err) {
31         perror("bind()");
32         exit(-1);
33     }
34     // 4. 将主动套接字转变为被动套接字
35     err = listen(sockfd, BACKLOG);
36     if (err) {
37         perror("listen()");
38         exit(-1);
39     }
40     cout << "Server is listening..." << endl;
41     return sockfd;
42 }
43
44 int main()
45 {
46     // 1. 绑定通配符地址, 监听 8888 端口
47     int listenfd = tcp_listen(8888);
48     // 2. 创建 epoll 实例 (红黑树, 双向链表)
49     int epfd = epoll_create(1);
50     if (epfd == -1) {
51         perror("epoll_create()");
52         exit(-1);
53     }
54     // 3. 将 listenfd 添加到 epoll 实例中 (监听)
55     struct epoll_event evt;
56     evt.events = EPOLLIN;
57     evt.data.fd = listenfd;
58     int err = epoll_ctl(epfd, EPOLL_CTL_ADD, listenfd, &evt);
59     if (err) {
60         perror("Add listenfd to epoll instance");
61         exit(-1);
62     }
63     // 4. 事件循环
64     for (;;) {
65         struct epoll_event events[MAXEVENTS]; // 存放就绪事件
66         // -1: 一直等待, 直到有事件就绪或发生错误
67         int nready = epoll_wait(epfd, events, MAXEVENTS, -1);
68         if (nready < 0) {
69             perror("epoll_wait()");
70             break;
71         }
72         // 遍历所有已就绪的事件

```

```

73     for (int i = 0; i < nready; ++i) {
74         if ((events[i].events & EPOLLIN) == 0) {
75             continue;
76         }
77         // a. listenfd就绪: 有新连接 (事件 1)
78         if (events[i].data.fd == listenfd) {
79             struct sockaddr_in cliaddr; // 用于接收客户端地址
80             socklen_t length = sizeof(cliaddr);
81             int connfd = accept(listenfd, (struct sockaddr*)&cliaddr, &length);
82             if (connfd == -1) {
83                 perror("accept()");
84                 continue;
85             }
86             // 将 connfd 添加到 epoll 实例中 (监听)
87             evt.events = EPOLLIN;
88             evt.data.fd = connfd;
89             epoll_ctl(epfd, EPOLL_CTL_ADD, connfd, &evt);
90             // 打印日志信息
91             char ipstr[INET_ADDRSTRLEN];
92             inet_ntop(AF_INET, &cliaddr.sin_addr, ipstr, INET_ADDRSTRLEN);
93             uint16_t port = ntohs(cliaddr.sin_port);
94             printf("新连接建立, 客户端地址为 %s:%hu\n", ipstr, port);
95         } else {
96             // 可能客户端有数据到达, 也可能是客户端断开了连接
97             char buf[4096];
98             int connfd = events[i].data.fd;
99             int nbytes = recv(connfd, buf, sizeof(buf), 0);
100             if (nbytes < 0) {
101                 // 发生错误
102                 perror("read()");
103                 // 将 connfd 从 epoll 实例中删除 (不再监听)
104                 epoll_ctl(epfd, EPOLL_CTL_DEL, connfd, NULL);
105                 close(connfd);
106             } else if (nbytes == 0) {
107                 // b. 客户端断开连接 (事件 2)
108                 // 将 connfd 从 epoll 实例中删除 (不再监听)
109                 epoll_ctl(epfd, EPOLL_CTL_DEL, connfd, NULL);
110                 close(connfd);
111                 printf("客户端断开连接, 关闭文件描述符%d\n", connfd);
112             } else {
113                 // c. 客户端有消息到达 (事件 3)
114                 for (int j = 0; j < nbytes; ++j) {
115                     buf[j] = toupper(buf[j]); // 将字母转为大写
116                 }
117                 send(connfd, buf, nbytes, 0);
118                 // d. 消息发送完毕 (事件 3.5, 这只能算半个事件)
119                 printf("%.s", nbytes, buf);
120             }
121         }
122     }
123 }

```

```
124
125 end:
126     close(listenfd);
127     return 0;
128 }
```

原生 TCP 编程结合 I/O 多路复用（如 `epoll/select`）的核心弊端是编程复杂度极高、代码维护困难且容易出错。主要体现在以下两点：

1. 编程复杂度高：直接使用 `epoll` 需要开发者手动维护 `socket` 的状态、连接、缓冲区等。
2. IO 线程与业务逻辑混杂：如果在处理 `epoll_wait()` 的循环中执行耗时的业务逻辑，会导致整个线程阻塞，从而无法及时响应其他描述符的读写事件，导致性能下降。

Reactor 模式本质上是对网络编程和 I/O 多路复用技术的面向对象封装和模式化实现。它有以下优点：

1. 事件驱动解耦：Reactor 将 `socket` 读写、连接接收等行为转变为“事件”驱动，主线程专注于“事件分发”，不仅保证了高性能，还让代码结构清晰。
2. 提升吞吐量：Reactor 模式允许将 I/O 的读写与具体业务逻辑解耦，耗时的业务逻辑可以交由工作线程池处理，避免了单一线程的阻塞。
3. 处理标准化：屏蔽了底层 `epoll/poll` 的繁琐细节（如缓冲区状态检查），提供统一的事件回调接口。
4. 支持高并发：通过 Reactor 机制能够实现单线程管理成千上万个连接，相比传统阻塞式编程极大地节省了系统线程资源。

总结

原生 I/O 多路复用提供了高性能的“可能”，而 Reactor 模式则将这种可能转化为稳定、可维护、易于开发的高并发网络框架。

1.4 TCP 编程的三个半事件

从上面 `EchoServer` 这个简单的例子中，我们也可以看出：TCP 编程最重要的就是处理好它的三个半事件。

1. 连接的建立：包括服务端接受 (`accept`) 新连接和客户端成功发起 (`connect`) 连接。TCP 连接一旦建立, 客户端和服务端是平等的, 可以各自收发数据。
2. 连接的断开：包括主动断开 (`close`、`shutdown`) 和被动断开 (`read` 返回 0)。
3. 消息到达，文件描述符可读：这是最为重要的一个事件，对它的处理方式决定了网络编程的风格（阻塞还是非阻塞，如何处理分包，应用层的缓冲如何设计，等等）。
4. 消息发送完毕：这算半个事件，对于低流量的服务，可以不必关心这个事件。另外，这里的“发送完毕”是指将数据写入操作系统的缓冲区，将由 TCP 协议栈负责数据的发送与重传, 不代表对方已经收到数据。

为什么消息发送完毕只能算半个事件？

原因在于：TCP 协议栈本身没有提供“对端已成功接收并处理数据”的确认机制。应用程序通常关心的“发送完成”有两种截然不同的含义：

1. 本机写入成功（半个事件）
调用 `send()/write()` 返回成功，仅代表数据已从应用程序缓冲区拷贝到内核的 TCP 发送缓冲区，并不代表数据已经通过网络到达对端，更不代表对端应用程序已读取。它只是“发送动作完成”，而不是“对方接收成功”。
2. 对端应用程序接收成功（完整事件）

应用层真正想要的事件往往是“对方已收到并处理”。但这需要设计应用层协议的确认机制。TCP 的 ACK 只保证对方 TCP 栈收到，不保证对端应用程序接收成功。
操作系统无法提供一个标准事件来通知“应用层数据已被对端应用程序接收”，因此只能算半个事件。

2 Reactor 模式

2.1 简介

Reactor 模式，又称反应器模式，是一种事件驱动的并发编程模型。它通过将事件分发机制与事件处理逻辑解耦，实现高性能、可扩展的并发处理。其核心思想在于利用操作系统提供的 I/O 多路复用机制（如 **select**、**poll**、**epoll** 等），高效地监听和分发事件，使得程序能够以非阻塞方式处理多个 I/O 事件。

Reactor 模式的实现通常依赖于操作系统提供的 I/O 多路复用机制，如 **select**、**poll**、**epoll** 等。这些机制允许同时监视多个文件描述符或套接字，当其中一个文件描述符或套接字准备好进行 I/O 操作时，多路复用机制会通知应用程序。

Reactor 模型在高性能网络编程中是绝对的主流，几乎无处不在。

- **libevent/libev**: C 语言领域最著名的事件库，Nginx/Tmux/Transmission 等无数项目基于它们。
- **Netty**: Reactor 模式的教科书级实现。Doug Lee 的《Scalable IO in Java》就是它的理论蓝本。绝大多数 Java 互联网公司的底层 RPC 框架都基于它。
- **Nginx**: 经典的主从 Reactor。Master 进程是“管理者”，Worker 进程（每个都是一个独立 Reactor）处理事件，它支撑着全球大量高并发网站。
- **Redis**: 单 Reactor 单线程的极致典范。它用简单模型实现了极高吞吐量（得益于内存操作极快）。
- **Kafka**: 内部网络层采用了基于 Reactor 的 Selector 机制，处理海量生产者和消费者的连接请求。

2.2 分类

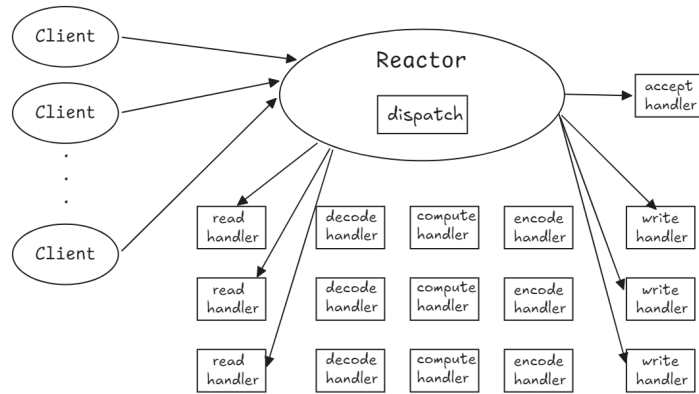
Reactor 模式根据事件的分发方式和多线程的结合程度，通常分为 3 种类型：

- 单 Reactor 单线程
- 单 Reactor 多线程
- 多 Reactor 多线程（Multi-Reactor）

这三个模型的区别在于：谁来监听事件、谁来处理读写、谁来执行业务逻辑。

2.3 单 Reactor 单线程

单 Reactor 单线程的模型如下图所示：



工作流程：

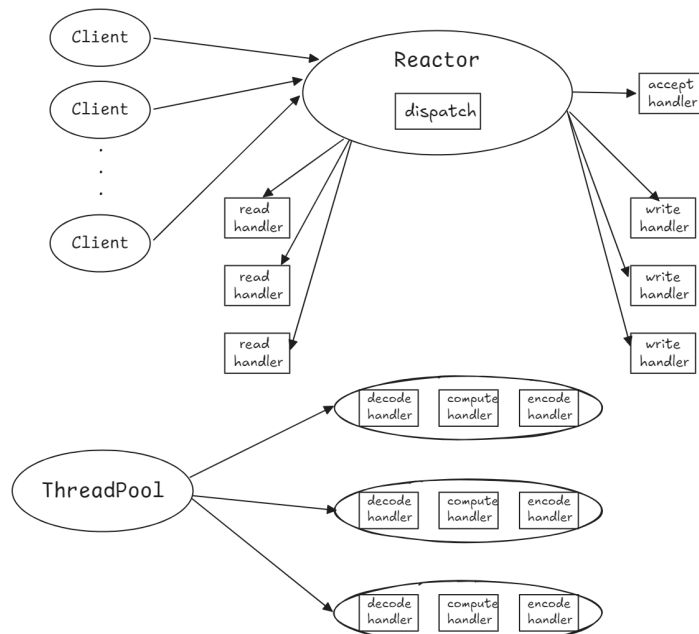
- Reactor 线程¹ 通过 I/O 多路复用监听所有的套接字（监听套接字 + 连接套接字）。
- 监听套接字读事件就绪：调用 **accept handler** 接受新连接，将新连接的套接字注册到 Reactor 中。
- 连接套接字读事件就绪：客户端有数据到达，调用 **read handler** 读取数据 → 解码 → 业务计算 → 编码。
- 写事件就绪：套接字发送缓冲区未满，执行 **write handler** 发送响应。

在单 Reactor 单线程模型中，所有的事情都是在一个线程中完成的。

优点	缺点
模型简单，没有并发问题（单线程）	性能瓶颈：一个线程做所有事，CPU 密集型业务会阻塞 I/O
编码容易，无需考虑共享数据同步	无法利用多核 CPU
适合业务逻辑极简单的场景	单点故障：Reactor 线程挂掉则服务不可用

2.4 单 Reactor 多线程

单 Reactor 多线程的模型如下图所示：



¹ Reactor 线程：执行 `EventLoop.loop()` 的那个线程

工作流程：

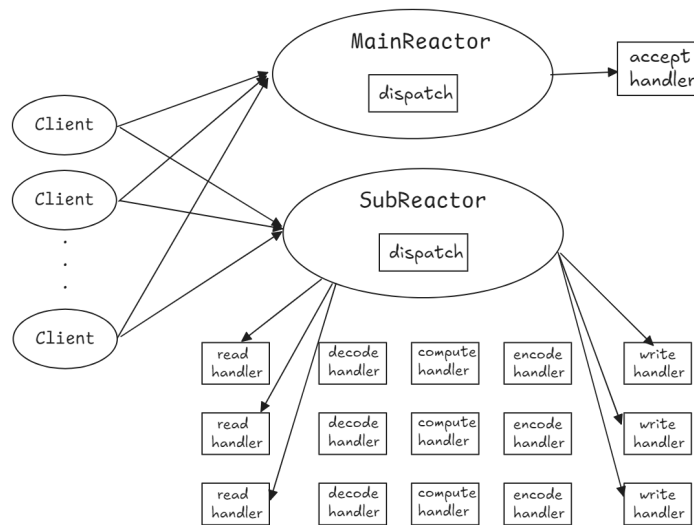
- **Reactor** 线程通过 **I/O** 多路复用监听所有的套接字（监听套接字 + 连接套接字）。
- 监听套接字读事件就绪：调用 **accept handler** 接受新连接，将新连接的套接字注册到 **Reactor** 中。
- 连接套接字读事件就绪：客户端有数据到达，调用 **read handler** 读取数据，然后交给 **Worker** 线程池处理。
- **Worker** 线程执行解码、业务计算、编码响应操作。
- 写事件就绪：套接字发送缓冲区未满，执行 **write handler** 发送响应。
- 在 **Worker** 线程执行解码、业务计算、编码响应时，**Reactor** 线程不会被阻塞，依然能够监听事件。

Reactor 只负责监听事件和 **I/O** 的读写（非阻塞的轻量操作），业务逻辑交给线程池执行（可能阻塞的重操作）。

优点	缺点
充分利用多核 CPU 处理业务逻辑	Reactor 单线程仍是瓶颈（大量连接的 I/O 事件监听）
业务逻辑阻塞不影响 I/O 事件监听	高并发下， Reactor 线程可能因事件分发过多而繁忙
相比单线程模型，吞吐量提升明显	编码复杂：需要处理线程安全问题

2.5 主从 Reaactor 多线程 (Multi-Reactor)

主从 **Reactor** 的模型如下图所示：



工作流程：

- **MainReactor** 线程通过 **I/O** 多路复用监听 **listenfd**。
- 监听套接字读事件就绪：新连接到来，**MainReactor** 接受新连接，并将新连接的套接字随机或轮询地分配给某个 **SubReactor** 负责。
- 每个 **SubReactor** 是一个独立的线程，负责监听自己管理的那一组连接的读写事件。
- 连接套接字的读事件就绪：客户端有数据到达，**SubReactor** 调用 **Handler** 读取数据 → 解码 → 业务计算 → 编码。
- 写事件就绪：套接字发送缓冲区未满，**SubReactor** 调用 **write handler** 发送响应。

通常情况下，我们会将 **MainReactor** 的数量设置为 **1**，**SubReactor** 的数量设置为 **CPU** 核心数或 **CPU** 核心数的两倍。一个连接固定由某一个 **SubReactor** 处理，有效地避免了并发问题。各个 **SubReactor** 之间是相互独立的，不需要竞争锁。

优点	缺点
彻底解决单 Reactor 的性能瓶颈	模型最复杂，实现难度高
MainReactor 专注于接受新连接，不会被读写操作阻塞	线程数较多，有上下文切换的开销
SubReactor 可线性扩展（增加线程）	需要合理配置线程数
高并发下性能最优	调试和问题定位更困难

3 muduo 网络库

3.1 简介

muduo 是一个基于 C++11 开发的高性能事件驱动网络库，专为编写高并发 TCP 服务器而设计。它采用了现代 C++ 的设计理念和新技术，结合了高效的事件驱动机制、多线程编程以及零拷贝技术，使得开发者能够轻松构建出高性能、可扩展的网络应用。

muduo 的核心设计基于主从 **Reactor** 模式和非阻塞 **IO**，这使得它能够在处理大量并发连接时表现出色。主 **Reactor** 负责监听新的连接请求，并将新连接分配给从 **Reactor** 进行处理。从 **Reactor** 则专注于处理已建立连接上的 **IO** 事件，从而实现了高效的事件分发和处理。

此外，**muduo** 还提供了丰富的功能和特性，如线程池、定时器、缓冲区管理等，这些功能极大地简化了网络编程的复杂性，提高了开发效率：

- 主从 **Reactor** 架构：主 **Reactor** 负责处理新连接的建立，从 **Reactor** 负责处理已连接的 **IO** 事件。这种架构能够充分利用多核 CPU 的优势，提高服务器的并发处理能力。
- 线程池机制：通过固定大小的线程池来处理任务，实现了任务的异步执行和负载均衡，避免了频繁创建和销毁线程带来的开销。
- 零拷贝设计：使用 **Buffer** 类来管理数据缓冲区，通过巧妙的设计实现了数据的零拷贝传输，减少了数据拷贝的次数，提高了数据传输效率。
- 定时器支持*：基于 **timefd** 实现了高精度的定时器，能够满足各种定时任务的需求，如心跳检测、超时处理等。

3.2 安装

muduo 依赖 **boost** 库，所以我们需要先安装 **boost**：

```
# 更新软件包列表，获取最新的可用版本信息
sudo apt-get update
# 安装 Boost C++ 库的全套开发文件（头文件及预编译库）
sudo apt-get install libboost-all-dev
```

之后，按照 **CMake** 项目的标准步骤安装即可：

```
# 进入项目源代码目录
cd muduo
# 创建并进入构建目录（用于存放编译中间文件和结果）
mkdir build && cd build
# 运行 CMake 配置项目，不编译示例程序
cmake -DMUDUO_BUILD_EXAMPLES=OFF ..
# 编译项目，生成可执行文件和库文件
make
# 将编译好的文件安装到系统目录（如 /usr/local/bin、/usr/local/lib）
```

```
sudo make install
# 更新动态链接库缓存，使系统能找到新安装的库文件
sudo ldconfig
```

安装成功之后，`/usr/local/lib` 目录下会多四个静态库：`libmuduo_base.a`、`libmuduo_net.a`、`libmuduo_http.a` 和 `libmuduo_inspect.a`；`/usr/local/include` 目录下会多一个文件夹 `muduo`：里面存放了 `muduo` 库的所有头文件。

接下来，我们通过三个经典的示例（`daytime` 服务器、`echo` 服务器、`chat` 服务器）来学习 `muduo` 网络库。如果这三个示例，你都能搞懂，并独立地编写，那么你就可以使用 `muduo` 库完成绝大多数网络编程的任务了。

3.3 daytime 服务器

```
                                     daytime.h
1  #pragma once
2
3  #include <muduo/base/Timestamp.h>
4  #include <muduo/net/Callbacks.h>
5  #include <muduo/net/InetAddress.h>
6  #include <muduo/net/TcpConnection.h>
7  #include <muduo/net/TcpServer.h>
8
9  // RFC 867
10 // 功能：当客户端连接时，服务器返回当前的日期时间字符串，然后关闭连接。
11 // 如果客户端发送数据，服务器会接收并丢弃（忽略）。
12 class DaytimeServer {
13 public:
14     // 构造函数
15     // @param loop: 事件循环 (EventLoop)，用于处理网络事件
16     // @param listenAddr: 服务器监听的地址和端口
17     DaytimeServer(muduo::net::EventLoop* loop,
18                 const muduo::net::InetAddress& listenAddr);
19
20     // 启动服务器（开始监听）
21     void start();
22
23 private:
24     // 连接回调函数：当新连接建立或断开时被调用
25     void onConnection(const muduo::net::TcpConnectionPtr& conn);
26
27     // 消息回调函数：当收到客户端数据时被调用
28     void onMessage(const muduo::net::TcpConnectionPtr& conn,
29                  muduo::net::Buffer* buf,
30                  muduo::Timestamp time);
31
32 private:
33     muduo::net::TcpServer server_; // TCP 服务器对象，管理监听和连接
34 };
```

```
                                     daytime.cc
1  #include "daytime.h"
2
3  #include <muduo/base/Logging.h> // 日志功能
```

```

4 #include <muduo/base/Timestamp.h> // 时间戳类
5 #include <muduo/net/Callbacks.h> // 回调函数类型定义
6 #include <muduo/net/EventLoop.h> // 事件循环
7
8 using namespace muduo;
9 using namespace muduo::net;
10
11 // 构造函数实现
12 DaytimeServer::DaytimeServer(EventLoop* loop, const InetAddress& listenAddr)
13     : server_(loop, listenAddr, "DaytimeServer") // 初始化 TcpServer
14 {
15     // 注册连接回调函数：当连接建立或关闭时，调用 onConnection
16     server_.setConnectionCallback(std::bind(&DaytimeServer::onConnection, this, _1));
17
18     // 注册消息回调函数：当收到数据时，调用 onMessage
19     server_.setMessageCallback(std::bind(&DaytimeServer::onMessage, this, _1, _2, _3));
20 }
21
22 // 启动服务器（内部调用 TcpServer 的 start 方法）
23 void DaytimeServer::start()
24 {
25     server_.start();
26 }
27
28 // 连接回调函数实现
29 void DaytimeServer::onConnection(const TcpConnectionPtr& conn)
30 {
31     // 记录连接信息：客户端地址 -> 服务器地址，状态 UP 或 DOWN
32     LOG_INFO << "DaytimeServer - " << conn->peerAddress().toIpPort() << " -> "
33         << conn->localAddress().toIpPort() << " is "
34         << (conn->connected() ? "UP" : "DOWN");
35
36     // 如果连接已建立（UP 状态）
37     if (conn->connected()) {
38         // 发送当前时间的字符串格式（例如 "2025-04-08 12:34:56.789\n"）
39         conn->send(Timestamp::now().toFormattedString() + "\n");
40
41         // 只关闭写端，服务器进入半连接状态，依然可以读取客户端的数据
42         // conn->shutdown();
43         // 主动关闭连接（Daytime 协议：发送完时间后服务器关闭连接）
44         conn->forceClose();
45     }
46 }
47
48 // 消息回调函数实现
49 void DaytimeServer::onMessage(const TcpConnectionPtr& conn, Buffer* buf, Timestamp time)
50 {
51     // 读取缓冲区中的所有数据（无论客户端发来什么）
52     string msg(buf->retrieveAllAsString());
53
54     // 记录日志：丢弃了多少字节的数据，以及收到数据的时间

```

```

55     LOG_INFO << conn->name() << " discards " << msg.size()
56         << " bytes received at " << time.toString();
57 }

```

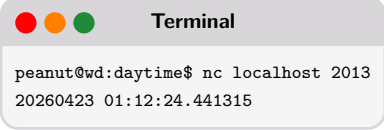
```

                                main.cc
1 #include "daytime.h"
2 #include <muduo/net/EventLoop.h> // 事件循环
3 #include <muduo/net/InetAddress.h> // 网络地址封装
4
5 using namespace muduo;
6 using namespace muduo::net;
7
8 int main()
9 {
10     // 1. 创建事件循环 (Reactor 核心)
11     EventLoop loop;
12     // 2. 创建监听地址: 监听所有网卡 (0.0.0.0) 的 2013 端口
13     InetAddress listenAddr(2013);
14     // 3. 创建 Daytime 服务器, 传入事件循环和监听地址
15     DaytimeServer server(&loop, listenAddr);
16     // 4. 启动服务器 (开始监听)
17     server.start();
18     // 5. 进入事件循环 (阻塞运行, 直到 loop 被停止)
19     loop.loop();
20     return 0; // 正常情况下不会执行到这里 (除非 loop 退出)
21 }

```

编译: `g++ main.cc daytime.cc -lmuduo_base -lmuduo_net -lpthread -o daytime`

使用 `nc` 命令连接服务器: `nc localhost 2013`, 执行结果如下:



```

Terminal
peanut@wd:daytime$ nc localhost 2013
20260423 01:12:24.441315

```

讲解

创建服务器的流程:

`daytime` 服务器虽然功能简单: 当客户端连接时, 服务器返回当前的日期时间字符串, 然后关闭连接 (短链接)。但它很好地展示了如何使用 `muduo` 创建 TCP 服务器:

1. `TcpServer server(&loop, listenAddr, "Daytime")`: 创建 TCP 服务器 (需要传入事件循环 `EventLoop`、网络地址 `InetAddress` 和服务器的名字)。
2. `server.start()`: 启动服务器 (监听端口, 并将监听套接字注册到 `EventLoop` 中)。
3. `loop.loop()`: 执行事件循环, 监听事件。

回调函数:

1. `server_.setConnectionCallback`: 注册连接回调函数, 当连接建立或关闭时会调用注册的函数。
2. `server_.setMessageCallback`: 注册消息回调函数, 当收到数据时会调用注册的函数。
3. `server_.setWriteCompleteCallback`: 注册写完回调函数: 写完消息时会调用注册的函数。一般我们不设置。

很明显，这些回调函数对应着我们前面讲过 TCP 编程的三个半事件。

日志：

muduo 库的日志系统提供了 6 种标准的日志级别，按严重程度从低到高排列如下：

日志级别	作用与说明
TRACE	级别最低，用于输出比 DEBUG 更细粒度的追踪信息，通常在开发调试时使用。
DEBUG	输出详细的调试信息，帮助开发人员在开发阶段定位问题，生产环境通常关闭。
INFO	输出程序运行的关键流程信息，这是 muduo 库默认的日志级别。
WARN	输出警告信息，表示程序可能出现了潜在问题，但系统仍能正常运行。
ERROR	输出错误信息，表示发生了一个错误事件，但程序可以继续运行。
FATAL	级别最高，表示发生了致命的错误，程序在输出此级别日志后会调用 abort() 异常退出。

在实际编程中，我们不需要关心底层的日志对象是如何构造的，直接使用这些对应的宏即可。这些宏使用起来和 std::cout 一样方便：

```
LOG_TRACE << "这是追踪信息，只有开启 TRACE 级别才会输出";
LOG_DEBUG << "这是调试信息，id=" << 9527;
LOG_INFO << "服务器已启动，监听端口：8080";
LOG_WARN << "警告：磁盘使用率超过 80%";
LOG_ERROR << "写入数据库失败，错误码：" << error_code;
LOG_SYSERR << "日志级别为 ERROR，会输出 strerror(errno)，系统调用出错时使用";
LOG_FATAL << "出现致命错误时使用，程序会调用 abort() 异常退出";
LOG_SYSFATAL << "出现致命错误时使用，日志级别为 FATAL，会输出 strerror(errno)";
```

此外，我们还可以通过 Logger::setLogLevel() 来动态调整日志的输出级别，从而控制日志的输出：

```
// 设置为 WARN 级别，将只输出 WARN、ERROR 和 FATAL 级别的日志
muduo::Logger::setLogLevel(muduo::Logger::WARN);
```

3.4 echo 服务器

```
----- echo.h -----
1 #pragma once
2
3 #include <muduo/net/TcpServer.h> // muduo 的 TCP 服务器类
4
5 // RFC 862 定义了 Echo 协议：服务器将客户端发送的数据原样返回
6 class EchoServer {
7 public:
8     // 构造函数
9     // loop: 事件循环 (EventLoop)，每个 TcpServer 需要绑定到一个 EventLoop
10    // listenAddr: 监听的 IP 和端口地址
11    EchoServer(muduo::net::EventLoop* loop,
12              const muduo::net::InetAddress& listenAddr);
13
14    // 启动服务器
15    void start();
16
17 private:
```



```

18 // 连接建立或关闭时的回调函数
19 void onConnection(const muduo::net::TcpConnectionPtr& conn);
20
21 // 收到消息时的回调函数
22 void onMessage(const muduo::net::TcpConnectionPtr& conn,
23               muduo::net::Buffer* buf,
24               muduo::Timestamp time);
25
26 // muduo 的 TCP 服务器对象, 负责网络 I/O 和事件分发
27 muduo::net::TcpServer server_;
28 };

```

```

                                echo.cc
1 #include "echo.h"
2 #include <muduo/base/Logging.h> // muduo 的日志模块
3
4 using std::placeholders::_1; // std::bind 的占位符
5 using std::placeholders::_2;
6 using std::placeholders::_3;
7
8 using namespace muduo;
9 using namespace muduo::net;
10
11 // 构造函数: 初始化 TcpServer
12 EchoServer::EchoServer(EventLoop* loop, const InetAddress& listenAddr)
13     : server_(loop, listenAddr, "EchoServer") // 传入 loop、地址、服务器名称
14 {
15     // 设置连接回调函数
16     server_.setConnectionCallback(std::bind(&EchoServer::onConnection, this, _1));
17     // 设置消息回调函数
18     server_.setMessageCallback(std::bind(&EchoServer::onMessage, this, _1, _2, _3));
19     // 设置 IO 线程池大小为 4, 即创建 4 个 sub-Reactor 线程
20     server_.setThreadNum(4);
21 }
22
23 // 启动服务器
24 void EchoServer::start()
25 {
26     server_.start(); // TcpServer::start() 开始监听
27 }
28
29 // 连接事件的回调
30 void EchoServer::onConnection(const TcpConnectionPtr& conn)
31 {
32     // 输出连接信息: 客户端地址 -> 本地地址 的状态 (UP/DOWN)
33     LOG_INFO << "EchoServer - " << conn->peerAddress().toIpPort() << " -> "
34              << conn->localAddress().toIpPort() << " is "
35              << (conn->connected() ? "UP" : "DOWN");
36 }
37
38 // 收到消息的回调

```

```

39 void EchoServer::onMessage(const TcpConnectionPtr& conn, Buffer* buf, Timestamp time)
40 {
41     // 将缓冲区中的所有数据取出为一个字符串
42     string msg(buf->retrieveAllAsString());
43     // 日志记录: 连接名、字节数、接收时间
44     LOG_INFO << conn->name() << " echo " << msg.size() << " bytes, "
45             << "data received at " << time.toString();
46     // 原样发送回客户端
47     conn->send(msg);
48 }

```

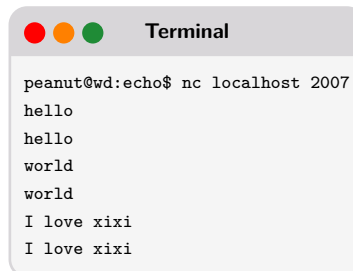
```

main.cc
1 #include "echo.h"
2 #include <muduo/base/Logging.h>
3 #include <muduo/net/EventLoop.h>
4
5 using namespace muduo;
6 using namespace muduo::net;
7
8 int main()
9 {
10     // 1. 创建事件循环对象 (Reactor 的核心)
11     EventLoop loop;
12     // 2. 监听的地址: 端口 2007, IP 为任意地址 (0.0.0.0)
13     InetAddress listenAddr(2007);
14     // 3. 创建 EchoServer 实例, 传入 loop 和地址
15     EchoServer server(&loop, listenAddr);
16     // 4. 启动服务器 (开始监听)
17     server.start();
18     // 5. 进入事件循环, 直到 loop 被退出
19     loop.loop();
20     return 0; // 正常情况下不会执行到这里
21 }

```

编译: `g++ main.cc echo.cc -lmuduo_base -lmuduo_net -lpthread -o echo`

使用 `nc` 命令连接服务器: `nc localhost 2007`, 执行结果如下:



```

Terminal
peanut@wd:echo$ nc localhost 2007
hello
hello
world
world
I love xixi
I love xixi

```

讲解

1. `echo` 服务器的逻辑和 `daytime` 服务器大同小异。不同点在于: 建立新连接后, `echo` 服务器不会立刻关闭连接 (长连接), 而是交给从 `Reactor` 监听和处理。

2. `muduo` 是主从 `Reactor` 模式的, 我们可以通过 `setThreadNum(int numThreads)` 函数设置处理客户端连接读写事件的 I/O 线程数目, 即从 `Reactor` 的数目。注意: `setThreadNum()` 必须在 `start()` 函数之前调用, 否则设置不会生效。

numThreads	线程模型	说明
0 (默认)	单线程模型	所有 I/O 操作（包括监听、读写）都在主线程 (main-Reactor) 中完成，不创建额外的 I/O 线程。
N > 0	多线程模型 (one loop per thread)	主线程负责接受新连接，线程池中将创建 N 个 I/O 线程 (sub-Reactor)，连接会通过轮询方式分配给 I/O 线程，实现负载均衡。

那 I/O 线程的数目该如何选择呢？通常会设置为 CPU 核心数² 或 CPU 核心数的 2 倍。

3.5 chat 服务器

下面只罗列了客户端和编解码器（codec）的代码，服务器的代码和前面两个示例大同小异。完整代码见 `examples/chat` 目录。

```

client.cc
1 #include "ChatClient.h"
2 #include <iostream>
3 #include <muduo/base/Logging.h>
4 #include <muduo/net/EventLoop.h>
5 #include <pthread.h>
6
7 using namespace std;
8 using namespace muduo;
9 using namespace muduo::net;
10
11 // 输入线程函数：独立线程读取用户输入
12 void* inputThreadFunc(void* args)
13 {
14     ChatClient* client = static_cast<ChatClient*>(args);
15     string line;
16     while (getline(cin, line)) { // 阻塞等待用户输入
17         if (line == "/quit") {
18             client->disconnect(); // 用户输入退出命令
19             break;
20         }
21         if (!line.empty()) {
22             client->send(line); // 发送用户输入的消息
23         }
24     }
25     return NULL;
26 }
27
28 int main(int argc, char* argv[])
29 {
30     if (argc < 3) {
31         cout << "Usage: " << argv[0] << " <server_ip> <port>" << endl;
32         return 1;
33     }
34     // 设置日志级别为 INFO

```

²注：get_nprocs() 函数可以获取 CPU 核心数。

```

35     Logger::setLogLevel(Logger::INFO);
36
37     // 创建事件循环（主线程运行）
38     EventLoop loop;
39     // 解析服务器地址和端口
40     InetAddress serverAddr(argv[1], static_cast<uint16_t>(atoi(argv[2])));
41     // 创建聊天客户端
42     ChatClient client(&loop, serverAddr);
43     // 连接服务器
44     client.connect();
45
46     // 启动输入线程，避免阻塞网络事件循环
47     pthread_t tid;
48     pthread_create(&tid, NULL, &inputThreadFunc, (void*)&client);
49
50     // 进入事件循环（阻塞，直到 loop->quit() 被调用）
51     loop.loop();
52
53     return 0;
54 }

```

ChatClient.h

```

1 #pragma once // 防止头文件重复包含
2
3 #include "LengthHeaderCode.h"
4 #include <muduo/net/TcpClient.h>
5 #include <muduo/net/TcpConnection.h>
6
7 class ChatClient {
8 public:
9     ChatClient(muduo::net::EventLoop* loop,
10               const muduo::net::InetAddress& serverAddr);
11
12     void connect(); // 连接服务器
13     void disconnect(); // 断开连接
14     void send(const std::string& message); // 发送消息
15
16 private:
17     void onConnection(const muduo::net::TcpConnectionPtr& conn); // 连接状态变化回调
18     void onEntireMessage(const muduo::net::TcpConnectionPtr&, // 收到完整消息回调
19                          const std::string& message,
20                          muduo::Timestamp);
21
22 private:
23     muduo::net::TcpClient client_; // TCP 客户端
24     muduo::net::EventLoop* loop_; // 事件循环指针
25     muduo::net::TcpConnectionPtr connection_; // 当前连接
26     LengthHeaderCode codec_; // 编解码器
27 };

```

ChatClient.cc

```

1 #include "ChatClient.h"

```

```

2 #include <iostream>
3 #include <muduo/base/Logging.h>
4 #include <muduo/net/EventLoop.h>
5
6 using namespace std;
7 using namespace muduo;
8 using namespace muduo::net;
9
10 ChatClient::ChatClient(EventLoop* loop, const InetAddress& serverAddr)
11     : client_(loop, serverAddr, "ChatClient") // 初始化 TCP 客户端
12     , loop_(loop)
13     , codec_(std::bind(&ChatClient::onEntireMessage, this, _1, _2, _3))
14 {
15     // 注册连接回调: 当与服务器的连接建立或断开时调用
16     client_.setConnectionCallback(
17         std::bind(&ChatClient::onConnection, this, _1));
18     // 注册消息回调: 使用编解码器处理接收到的数据
19     client_.setMessageCallback(
20         std::bind(&LengthHeaderCodec::onMessage, &codec_, _1, _2, _3));
21 }
22
23 void ChatClient::connect()
24 {
25     client_.connect(); // 发起连接到服务器
26 }
27
28 void ChatClient::disconnect()
29 {
30     client_.disconnect(); // 断开与服务器的连接
31 }
32
33 void ChatClient::send(const string& message)
34 {
35     if (connection_) {
36         codec_.send(connection_, message); // 通过编解码器发送消息
37     }
38 }
39
40 void ChatClient::onConnection(const TcpConnectionPtr& conn)
41 {
42     if (conn->connected()) {
43         // 连接建立成功, 保存连接指针
44         connection_ = conn;
45         LOG_INFO << "Connected to server: "
46             << conn->peerAddress().toIpPort();
47         cout << "=== Connected to chat server ===" << endl;
48         cout << "Type your message and press Enter to send." << endl;
49         cout << "Type '/quit' to exit." << endl;
50         cout << "======" << endl;
51     } else {
52         // 连接断开, 重置连接指针

```

```

53     connection_.reset();
54     LOG_INFO << "Disconnected from server";
55     cout << "=== Disconnected from server ===" << endl;
56     // 退出事件循环，程序结束
57     loop_->quit();
58 }
59 }
60
61 // 收到服务器广播的消息时的回调
62 void ChatClient::onEntireMessage(const TcpConnectionPtr& conn,
63     const string& message,
64     Timestamp receiveTime)
65 {
66     // 直接显示收到的消息（服务器已经添加了时间戳）
67     cout << message << endl;
68 }

```

LengthHeaderCode.h

```

1 #pragma once
2 #include <functional>
3 #include <muduo/base/Timestamp.h>
4 #include <muduo/net/Buffer.h>
5 #include <muduo/net/TcpConnection.h>
6
7 // 协议格式：| Length (4 字节网络字节序) | Value (Length 字节) |
8 // 这种格式解决了 TCP 粘包问题：先读取 4 字节长度，再读取对应长度的消息体
9 class LengthHeaderCode {
10 public:
11     // 消息处理回调函数类型
12     // 参数：连接指针、消息内容、接收时间戳
13     using MessageCallback = std::function<void(const muduo::net::TcpConnectionPtr&,
14         const std::string&,
15         muduo::Timestamp)>;
16
17     // 构造函数：保存消息处理回调
18     explicit LengthHeaderCode(const MessageCallback& cb)
19         : messageCallback_(cb)
20     {
21     }
22
23     // 编码：将消息打包成 [长度+内容] 格式发送
24     void send(const muduo::net::TcpConnectionPtr& conn, const std::string& value);
25
26     // 解码：从接收缓冲区解析出一条完整消息
27     void onMessage(const muduo::net::TcpConnectionPtr& conn,
28         muduo::net::Buffer* buf,
29         muduo::Timestamp receiveTime);
30
31 private:
32     MessageCallback messageCallback_; // 解析完成后的回调函数
33 };

```

```

1 #include "LengthHeaderCode.h"
2 #include <endian.h> // 提供字节序转换函数
3
4 using namespace muduo;
5 using namespace muduo::net;
6
7 // 编码并发送消息
8 void LengthHeaderCode::send(const TcpConnectionPtr& conn, const std::string& value)
9 {
10     Buffer buf; // muduo 的缓冲区, 自动管理内存
11     uint32_t length_be = htobe32(value.size()); // 主机字节序转网络字节序 (大端)
12     // 将长度头添加到缓冲区
13     buf.append(&length_be, sizeof(length_be));
14     // 将消息体添加到缓冲区
15     buf.append(value.data(), value.size());
16     // 通过 TCP 连接发送缓冲区中的数据
17     conn->send(&buf);
18 }
19
20 // 解码接收到的数据 (处理 TCP 粘包问题)
21 void LengthHeaderCode::onMessage(const TcpConnectionPtr& conn,
22     Buffer* buf,
23     Timestamp receiveTime)
24 {
25     // 循环处理, 因为缓冲区可能包含多条完整消息
26     while (buf->readableBytes() >= 4) { // 至少要有 4 字节才能读取长度
27         // 窥探 (不取出) 前 4 字节, 获取消息长度
28         const void* data = buf->peek();
29         uint32_t length = be32toh(*static_cast<const uint32_t*>(data)); // 网络字节序转主机
30
31         // 检查消息是否完整: 总数据量 = 4 字节头 + length 字节体
32         if (buf->readableBytes() - 4 < length) {
33             // 数据不完整, 等待下一次 TCP 数据到达
34             break;
35         }
36
37         // 跳过长度头 (4 字节)
38         buf->retrieve(4);
39         // 提取消息体
40         std::string value(buf->peek(), length);
41         // 跳过已处理的消息体
42         buf->retrieve(length);
43
44         // 好莱坞原则: Don't call me, I'll call you back.
45         // 拿到一条完整的消息, 调用业务层注册的回调函数
46         messageCallback_(conn, value, receiveTime);
47     }
48 }

```

讲解

1. 创建客户端的流程

1. 创建 `EventLoop`（事件循环）和 `InetAddress`（服务器地址）。
2. 创建 `TcpClient` 对象，并设置两个核心回调函数。
3. 调用 `connect()` 发起连接。
4. 启动循环：调用 `loop()` 方法，开始处理网络事件。

2. 创建一个新线程，读取用户输入

在 `client.cc` 中，我们创建了一个新线程来读取用户的输入，避免阻塞 `Reactor` 线程（主线程）。

3. 编解码器

我们设计了一个编解码器类 (`LengthHeaderCodec`)，专门用来解析协议，避免 TCP 的粘包和半包问题。这也是 `muduo` 网络库推荐的做法。